

Verwendung von Arrays

	eindimensional	zweidimensional
Deklaration:	`int[] zahlen;`	`Color[][] pixel;`
Initialisierung:	`zahlen = new int[20];`	`pixel = new Color[100][50];`
Zugriff auf ein Arrayelement:	`zahlen[5] = 10;` `zahlen[i] = 12;`	`pixel[4][7] = Color.BLACK;` `pixel[x][y] = Color.BLUE;`
Direkte Initialisierung:	`char[] buchstaben = {'i','n','f'};`	`double[][] faktoren = {{ 4.0, -1.2, 2.3}, { -2.1, 2.1, 0.0}};`

Achtung: Bei zweidimensionalen, direkt initialisierten Arrays gibt die erste Position die Zeile und die zweite die Spalte an: `faktoren[0][2]` hat im Beispiel oben den Wert 2,3.

Auftrag:

1. *Gib die notwendigen Java-Befehle an, um ...*

a) *ein Array zu deklarieren und zu initialisieren, um 35 Kommazahlen zu speichern. Welche Position hat das erste bzw. das letzte Element des Arrays?*

b) *ein zweidimensionales Array für ein Bild der Größe 800x350 Pixel zu deklarieren und zu initialisieren.*

c) *den 4. Wert des eindimensionalen Arrays auf 45,2 zu setzen und den 3. Wert auf 13,2.*

d) *den 5. Wert des eindimensionalen Arrays auf die Summe des 3. und des 4. Wertes zu setzen.*

e) *das Pixel an der Stelle x = 17 und y = 34 auf Rot zu setzen.*

f) *dem Pixel an der Stelle x = 123 und y = 12 die Farbe des Pixels an der Stelle x = 76 und y = 99 zuzuweisen.*

Folgender Java-Code (in Processing) kopiert die Pixel eines Bildes sowohl in ein zweidimensionales Array als auch in ein neues Bild.

```
int breite = originalbild.width;
int hoehe = originalbild.height;

pixelNeu = new color[breite][hoehe];
PImage neuesBild = createImage(breite, hoehe, RGB);

for (int x = 0; x < breite; x++) {
  for (int y = 0; y < hoehe; y++) {
    pixelNeu[x][y]= pixelAlt[breite-1-x][y];
    neuesBild.set(x, y, pixelNeu[x][y]);
  }
}
```

Die verschachtelte Zählschleife ermöglicht es, jedes Pixel des neuen Bildes zu definieren.

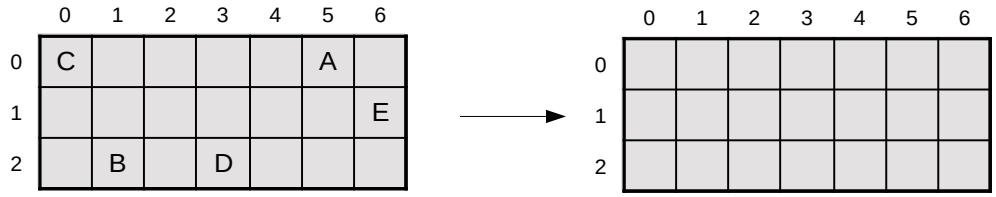
Auftrag:

2. *Gib an, in welcher Reihenfolge sie gesetzt werden, falls das Bild 7x3 Pixel groß ist:*

Durchgang	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
x																					
y																					

3. *Gib an, welches Pixel des Originalbildes an die Position (5|2) wandert. Woher kommt das Pixel mit der Position (0|0)? Wohin geht das Pixel, das ursprünglich an der Position (3|1) im Originalbild war?*

4. *Zeichne ein, an welcher Stelle die Pixel A- E im neuen Bild zu finden sind.*



5. *Beschreibe mit 3-4 Worten, wie das neue Bild aus dem alten hervorgeht.*

Programmierauftrag

6. *Öffne das Projekt "p01bildbearbeitung" in Processing. Starte das Projekt. Beschreibe, was in der setup()-Methode passiert.*

7. *Spiegle das geladene Bild horizontal. Rufe dazu in der setup()-Methode die Methode spiegleHorizontal auf: bildNeu = spiegelHorizontal(bildAlt). Nach dem Spiegeln musst du das neue Bild mit image() wieder anzeigen lassen. Achte darauf, dass es neben dem alten Bild angezeigt wird.*

8. *Speichere das neue Bild mit save(„data/spiegelsee.jpg“). Überprüfe im Projektordner, ob das Speichern geklappt hat.*

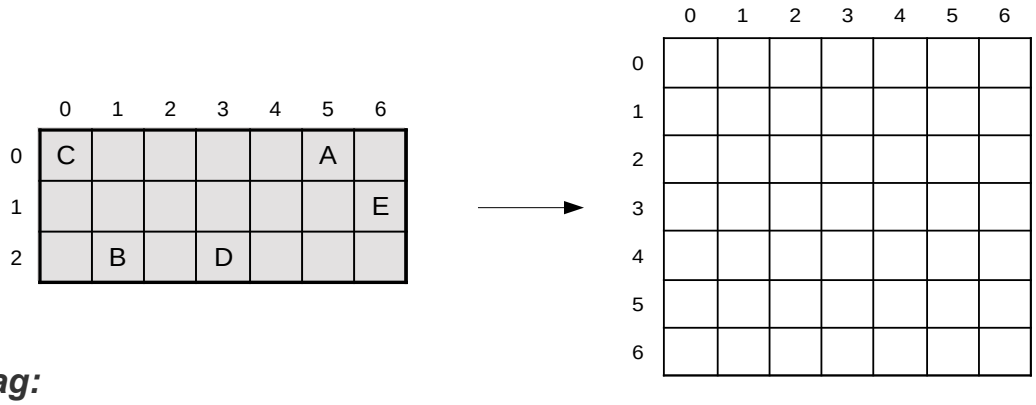
9. *Teste dein Programm auch mit einem anderen Bild aus dem data-Ordner.*

10. *Implementiere die Methode* `PImage spiegleVertikal(PImage originalbild)` *analog zur spiegleHorizontal-Methode.*

    Teste deine neue Methode.

Drehung des Bildes um 90°

Ein Bild soll um 90° nach links gedreht werden.



Auftrag:

11. *Markiere im Raster den Bereich, den das gedrehte Bild abdeckt. Wie kann die Breite und die Höhe des neuen Bildes bestimmt werden?*
12. *Zeichne ein, an welcher Stelle die Pixel A-E im neuen Bild zu finden sind.*
13. *Bestimme die Koordinaten der Punkte A-E im neuen Bild und im Originalbild.*

	A	B	C	D	E
x_{neu}	0				
y_{neu}	1				
$x_{original}$	5				
$y_{original}$	0				

14. *Ermittele Formeln, mit denen die Koordinaten des ursprünglichen Pixel bestimmt werden können:*

    $x_{original}$ =

    $y_{original}$ =

    In Java: `pixelNeu[x][y] = pixelAlt[ ][ ];`

Programmierauftrag:

15. *Implementiere die Methode* `PImage dreheLinks(PImage originalbild).`
16. *Stelle die gleichen Überlegungen (Aufgaben 11-14) für eine Drehung um 90° nach rechts an. Implementiere die Methode* `PImage dreheRechts(PImage originalbild).`
17. *Überlege, wie du mit den bestehenden Methoden eine Drehung um 180° realisieren könntest. Teste dies in Processing durch geeignete Methodenaufrufe. Implementiere eine Methode* `PImage drehe180(PImage originalbild)` *unter Verwendung der bestehenden Methoden.*
    Hinweis: das Zwischenergebnis darf auch als Bild angezeigt werden.

RGB-Farbmodell

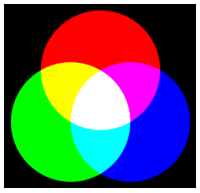


Abb. 1: Additive Farbmischung

Die Farbe eines Pixels kann man auf verschiedene Arten codieren. Eine davon ist das RGB-Farbmodell[1] (Rot-Grün-Blau):

Aufgabe:

1. *Ergänze die folgende Tabelle:*
Nutze dazu auch https://informatik.schule.de/rgb/RGB_farbmischer.html.

Farbe		Rot	Grün	Lila	Braun	Hellgrau	Dunkelgrau	Weiß	schwarz
Rotanteil	0-255	255							
	0.0-1.0	1.0							
Grünanteil	0-255	0							
	0.0-1.0	0.0							
Blauanteil	0-255	0							
	0.0-1.0	0.0							
Hexcode		FF0000							

2. *Beschreibe, wie du eine Farbe heller machen kannst, ohne den Farbton zu ändern.*

3. *Beschreibe, was passiert, wenn du die einzelnen Farbanteile einer Farbe allmählich aneinander annäherst.*

Farben in Java

Klasse color

Farben werden in Processing in Objekten der Klasse `color` gespeichert.

Farbe deklarieren:
```
color farbe;
```

Farbe initalisieren:
```
farbe = color(0,255,255);  // Cyan
farbe = #00FFFF
farbe = color(122); // Grauwert zwischen 0 und 255
```

Farbanteil ermitteln:
```
rotanteil = red(farbe); // blue(farbe),green(farbe)
```

Man kann einzelne Farbanteile einer Farbe nicht ändern, nur eine komplette neue Farbe initialisieren.

Aufgabe

4. *Deklariere zwei Variablen f1 und f2 vom Typ color in Processing. Initialisiere sie auf zwei verschiedene Arten, so dass sie die Farbe Lila repräsentieren.*

5. *Gib die notwendigen Befehle an, um f1 so zu verändern, dass der Blauanteil auf 0 und der Grünanteil auf 128 gesetzt wird. Gib an, welche Farbe nun repräsentiert wird.*

1 Abbildung 1: *Additivie Farbmischung*, *Quark67* (Lizenz: *CC BY-SA 3.0*) via Wikimedia Commons, URL: https://commons.wikimedia.org/wiki/File:Synthese%2B.svg.

Bildbearbeitungsalgorithmen 2: Umwandlung in Graustufenbilder



Bild Rose (eigenes Werk)

Minimum



Natürlich





Maximum



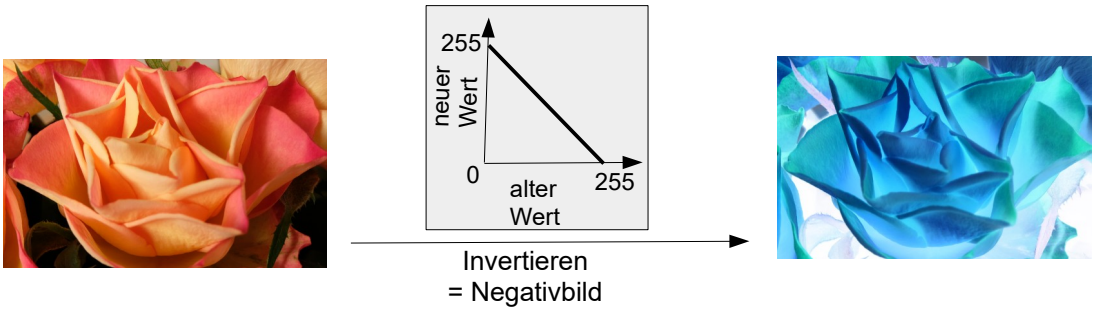
Durchschnitt

1. *Öffne die Datei p0farbmodell in Processing*.*
   Implementiere die Methode `PImage graustufenDurchschnitt(PImage originalbild)`*, die für jeden Pixel des Originalbildes den Mittelwert der Farbanteile berechnet und diesen als Helligkeit für die Graustufe nimmt.*

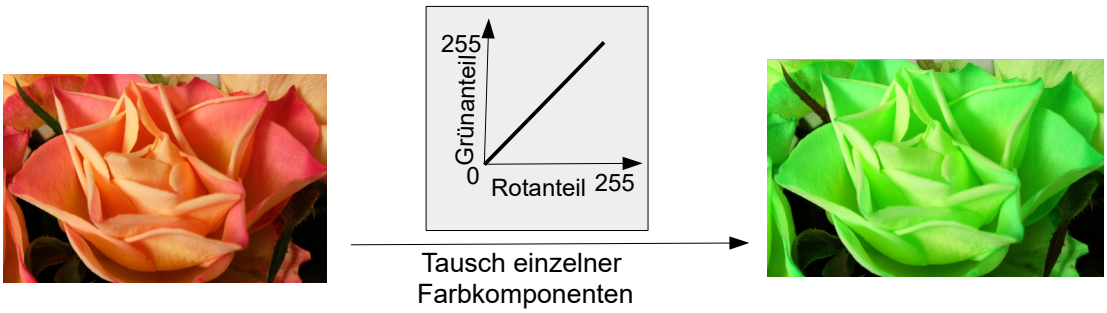
   - Mit Hilfe einer verschachtelten Zählschleife wird für jeden Pixel folgendes gemacht:
   - Hole Rot-, Grün- und Blauanteil aus dem Originalpixel heraus und speichere die Werte in lokalen Variablen.
   - Berechne den Durchschnitt m der drei Werte.
   - Übergebe den Durchschnitt als Grauwert (color(m)) dem Pixel des neuen Bildes.
   - Speichere diese im Pixel-Array des neuen Bildes.
   - Erstelle aus dem Pixel-Array ein neues Bild und gib es zurück.
2. *Implementiere analog zu* `graustufenDruchschnitt` *zwei weitere Methoden* `graustufenMin` *und* `graustufenMax`*, die das Minimum, bzw. Maximum der drei Farbanteile als Helligkeit der Graustufe nehmen.*
3. *Unser Auge ist nicht für alle drei Farben gleich empfindlich. Daher sollten die Farbanteile unterschiedlich gewichtet werden: Rot mit 29,9%, Grün mit 58,7% und Blau mit 11,4%. Implementiere eine Methode* `graustufenNatuerlich`*, die diese Vorgabe umsetzt.*

** Expert(inn)en können auch in einer neuen Processing-Datei starten. Dann müssen noch die beiden zweidimensionalen Arrays für das Originalbild und das neue Bild deklariert , die Pixel des Originalbildes in das entsprechende Array geschrieben und wieder die ersten Schritte in der setup()-Methode implementiert werden (Größe des Fensters, Hintergrundfarbe, Laden und Anzeigen des Bildes).*

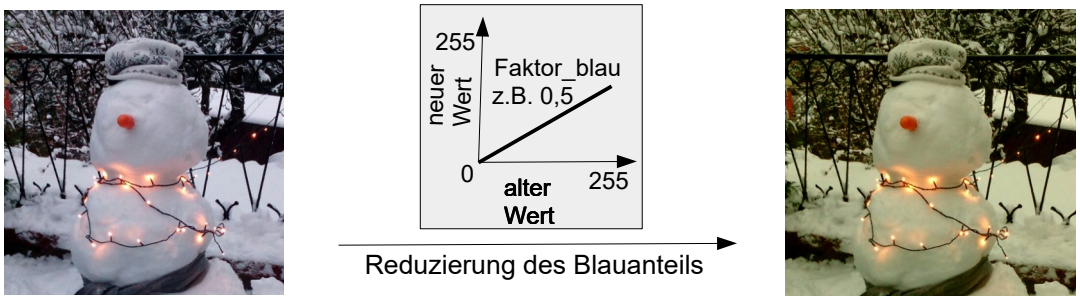
Bildbearbeitungsalgorithmen 3: RGB-Experimente



1. *Implementiere eine Methode* `invertiere`*, die alle Farbanteile umdreht, d.h. aus Rotanteil 0 wird Rotanteil 255 und umgekehrt. Man erhält ein Bild, das dem Negativbild entspricht, das bei der analogen Fotografie auf dem Film entstand.*

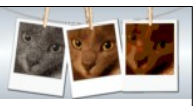


2. Implementiere eine Methode tauschRotGruen, die den Rot- und den Grünanteil jeder Farbe vertauscht. Analog kann man auch andere Farbkomponenten tauschen.



3. Implementiere eine Methode `PImage farbaenderung (PImage originalbild, double faktor_r, double faktor_g, double faktor_b)`, die jede Farbkomponente mit dem angegebenen Faktor multipliziert.

   Beachte dabei, dass durch die Multiplikation mit Faktoren größer als 1 oder kleiner als 0, Werte heraus kommen können, die nicht zum gültigen Wertebereich [0;255] gehören. Werte größer als 255 müssen auf 255 reduziert werden, Werte kleiner 0 müssen zu 0 werden.
4. Beschreibe, welchen Effekt es hat, wenn man die Methode mit farbaenderung(beispiel1, 0.0, 1.0, 1.0) aufruft. Was passiert bei farbaenderung(beispiel1, 0.5, 0.5, 0.5)?
5. Gib an, wie die Methode aufgerufen werden muss, um nur den Blauanteil eines Bildes zu bekommen. Gib an, wie die Methode aufgerufen werden muss, um ein Bild aufzuhellen.



Grundidee: Die umliegenden Pixel beeinflussen die Farbe eines Pixels

Für die Berechnung der Intensität eines neuen Pixels werden die Intensitäten der umliegenden Pixel mit einem festgelegten Faktor multipliziert und dann alle addiert. Dies wird als Faltung bezeichnet.

Hinweis: auf den Rand kann man den Filter nicht so einfach anwenden, wir lassen die Randpixel daher so wie sie sind.

Beispiel:

	0	1	2	3	4	5	6
0	1	3	2	1	2	3	1
1	2	3	2	1	2	3	2
2	2	3	3	1	2	3	3
3	3	4	4	2	5	4	3
4	6	2	3	8	5	2	1
5	2	1	2	2	1	1	2
6	2	1	2	3	4	2	1

Intensität der Originalpixel: I_{alt}

-1	-2	-1
0	0	0
1	2	1

Filtermatrix

	0	1	2	3	4	5	6
0	1	3	2	1	2	3	1
1	2	2					2
2	2						3
3	3						3
4	6					-11	1
5	2						2
6	2	1	2	3	4	2	1

Intensität der neuen Pixel: I_{neu}

Pixel an Position (1|1):

$$I_{neu}(1,1) = \begin{matrix} (-1)\cdot I_{alt}(0,0) & +(-2)\cdot I_{alt}(1,0) & +(-1)\cdot I_{alt}(2,0) \\ +(0)\cdot I_{alt}(0,1) & +(0)\cdot I_{alt}(1,1) & +(0)\cdot I_{alt}(2,1) \\ +(1)\cdot I_{alt}(0,2) & +(2)\cdot I_{alt}(1,2) & +(1)\cdot I_{alt}(2,2) \end{matrix} = -1-6-2+2+6+3 = 2$$

Pixel an Position (5|4):

$$I_{neu}(5,4) = \begin{matrix} (-1)\cdot I_{alt}(4,3) & +(-2)\cdot I_{alt}(5,3) & +(-1)\cdot I_{alt}(6,3) \\ +(0)\cdot I_{alt}(4,4) & +(0)\cdot I_{alt}(5,4) & +(0)\cdot I_{alt}(6,4) \\ +(1)\cdot I_{alt}(4,5) & +(2)\cdot I_{alt}(5,5) & +(1)\cdot I_{alt}(6,5) \end{matrix} = -5-8-3+1+2+2 = -11$$

Auftrag:

1. *Berechne die restlichen Werte des oben angegebenen Bildes.*
2. *Öffne die Datei 03_faltung.ods. Das Beispiel aus Aufgabe 1 ist ein Teil des Bsp. 1 (zufällige Verteilung).*
3. *Kontrolliere Deine Berechnung, indem du die Filtermatrix einträgst.*
4. *Untersuche, wie sich diese Filtermatrix auf die anderen Beispiele auswirkt. Beschreibe, wozu diese Filtermatrix verwendet werden könnte. Untersuche, welchen Effekt es hat, wenn du jeden Wert der Matrix verdoppelst*[1]*.*
5. *Untersuche genauso auch folgende Filtermatrizen:*

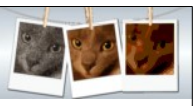
1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

0	-1	0
-1	4	-1
0	-1	0

0	-1	0
-1	5	-1
0	-1	0

-2	-1	0
-1	1	1
0	1	2

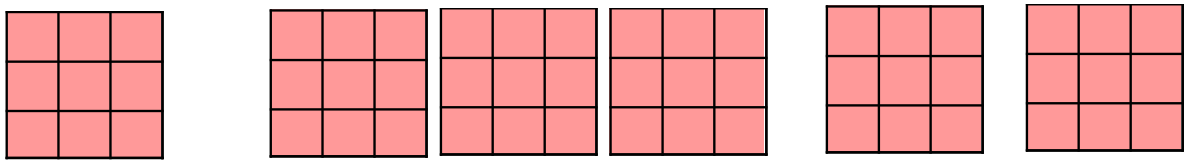
1 Man kann die Filtermatrizen auch in GIMP direkt an Bildern ausprobieren. Unter Filter->Generic->Convolution Matrix bzw. Filter->Allgemein->Faltungsmatrix kann man die Matrizen ausprobieren.



6. *Die Summe der Einträge der Matrix ist in der Regel 0 oder 1. Beschreibe, welchen Effekt es auf das einfarbige Bild hat, wenn die Summe 0 ist. Beschreibe, was passiert, wenn die Summe 1 ist. Untersuche, was passiert, wenn die Summe zwischen 0 und 1 liegt bzw. größer als 1 ist.*

Faltungsfilter

Glätten/Weichzeichnen | Kantenfinden | Schärfen | Relief



Bildbearbeitungsalgorithmen 4: Mehrpixeloperationen (Faltung)

Ziel: Es soll eine Methode `PImage faltung(PImage originalbild, float[][] filter)` implementiert werden, die den Filter auf ein Bild anwendet. Durch Aufruf mit geeigneten Filtern können dann die verschiedenen Bildbearbeitungsalgorithmen realisiert werden. Die Größe des Filters muss dabei nicht unbedingt 3x3 sein.

Programmierauftrag:

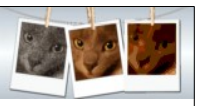
7. *Du erhältst ein Puzzle mit dem Quelltext der Methode. Ordne die Bauteile richtig an.*

8. *Kommentiere die einzelnen Abschnitte hinter den "//" - Kommentarzeichen. Folgende Fragen können dabei helfen. Du solltest auf jeden Fall Antworten auf die Fragen haben.*
   - *Welches Pixel hat die Koordinaten (x|y)? Welches Pixel hat die Koordinaten (xx|yy)?*
   - *Was speichern die Variablen laenge und halb?*
     Hinweis: Teilt man zwei ganze Zahlen durcheinander, werden die Nachkommastellen abgeschnitten: 3/2 = 1.
   - *Was gibt die Position (i | j) an?*
   - *Warum wird rot auf 0.0 gesetzt, wenn es kleiner als 0.0 ist? Warum auf 255.0, wenn rot größer als 255.0 ist?*
   - *Warum müssen die Variablen rot, gruen und blau als double definiert werden?*
   - *Warum beginnt die Schleife mit der Zählvariable x nicht bei 0, sondern bei halb?*

9. *Öffne die Datei p04faltung in Processing.*
   Füge dann die Methode faltung ein. Hinweis: Du kannst die Begrenzung eines Wertes auf den Bereich 0-255 in einer extra Methode auslagern, da dies mehrfach benötigt wird.

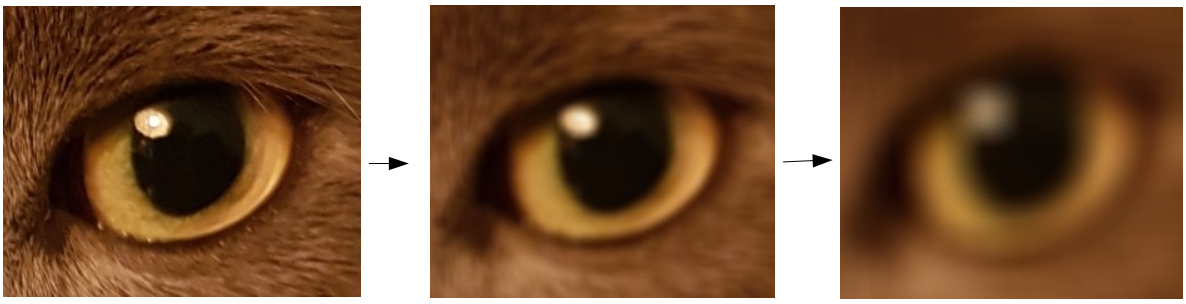
10. *Implementiere eine Methode* `PImage weichzeichnen(PImage originalbild).`
    Definiere dazu die Filtermatrix (vgl. direkte Initialisierung von Arrays) als zweidimensionales Array und rufe damit die Methode faltung auf.

11. *Implementiere analog die Methoden* `schaerfen`, `relief`, `kantenfindenHorizontal`, `kantenfindenVertikal` *und* `kantenfinden`.



Bildbearbeitungsalgorithmen 5: Größere Filtermatrizen

Die bisherigen Filtermatrizen waren immer 3x3 Pixel groß. Bei Bildern mit hoher Auflösung sieht man den Effekt kaum, wenn nur die direkt benachbarten Pixel herangezogen werden. Daher kann man die Filtermatrizen auch größer wählen.



11x11 Mittelwertfilter *31x31 Mittelwertfilter*

Weichzeichnen mit Mittelwertfilter:

Um das Weichzeichnen zu realisieren, haben wir eine Matrix mit identischen Werten in jeder Zelle verwendet. Die Summe ergab 1. Damit wird automatisch der Mittelwert der neun Pixel berechnet.

Diese Idee kann einfach auf größere Filter übertragen werden: Man bestimmt die Anzahl der betroffenen Pixel und trägt den Kehrwert in jede Zelle der Matrix ein.

Programmierauftrag:

1. *Implementiere eine Methode public Picture weichzeichnen(Picture originalbild, int groesse), die einen Mittelwertfilter der Größe groesse x groesse einsetzt.*

Tipps:

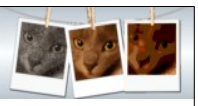
- *Deklariere die Matrix in der richtigen Größe.*
- *Berechne die Anzahl der Pixel der Matrix.*
- *Setze jeden Wert der Matrix auf 1.0/(Anzahl der Pixel), indem du zwei Schleifen verwendest.*
  ***Achtung:** 1.0/anzahl ist etwas anderes als 1/anzahl, da bei der ersten Rechnung eine Kommazahl berechnet wird, bei der zweiten eine ganzzahlige Division durchgeführt und der Rest abgeschnitten wird.*

Bei größeren Filtern bleibt am Bildrand ein sichtbarer Streifen von Pixeln, die nicht verändert werden, da der Filter über die Bildgrenze hinausragen würde. Möchte man dieses Problem beheben, gibt es drei Möglichkeiten:

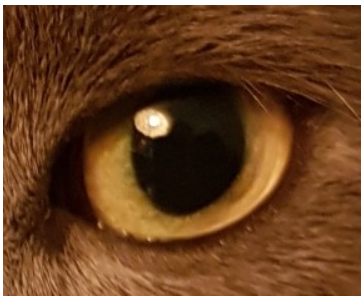
- Man setzt die Pixel außerhalb des Bildes alle auf "Schwarz".
- Man nimmt für die Pixel außerhalb des Bildes die Farbe des Pixels am Rand.
- Man spiegelt das Bild nach außen: z.B. bekommt das Pixel mit Koordinaten (-4|10) die Farbe des Pixels mit den Koordinaten (4|10).

Programmierauftrag:

2. *Passe die Implementierung der Methode faltung so an, dass eine drei Varianten umgesetzt wird. Überprüfe, ob das Bild nun wie gewünscht aussieht.*



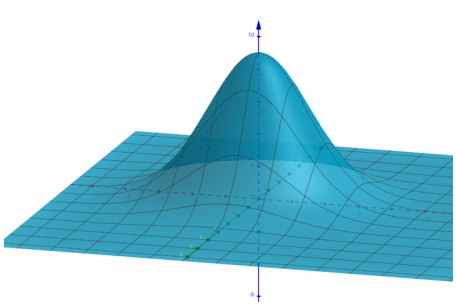
Bildbearbeitungsalgorithmen 6: Weichzeichnen mit Gaußfiltern





31x31 Gaußfilter

Bei größeren Filtern sollte der Einfluss weit entfernter Pixel geringer sein als der nahe liegender Pixel. Bei Mittelwertfilter haben alle Pixel den gleichen Einfluss. Der Gaußfilter behebt dieses Problem.



Gewichte des Gaußfilters

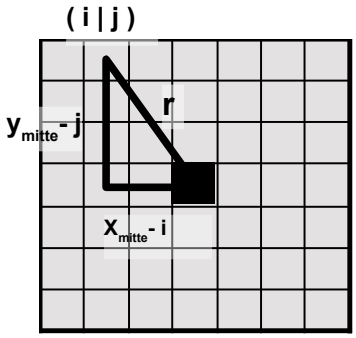
Nahe an der Mitte liegenden Zellen des Filters erhalten größere Werte als weiter außen liegende.

$$Gewicht(i,j) = K \cdot e^{-\frac{r^2}{2\sigma}}$$

Dabei ist r der Abstand vom mittleren Punkt des Gaußfilters und berechnet sich mit dem Pythagoras. Sigma (σ) ist die Standardabweichung der Gaußverteiltung und ergibt sich aus der Größe des Filters. Wählt man

$$\sigma = \frac{groesse}{6}$$

stellt man sicher, dass die Werte außerhalb des Filters nahezu keinen Einfluss auf die Berechnung hätten. K wird so gewählt, dass die Summe aller Gewichte 1 ergibt.



Programmierauftrag

1. *Implementiere eine Methode public Picture gaussfilter(Picture originalbild, int groesse), die einen Gaußfilter der Größe groesse x groesse generiert und für die Faltung verwendet.*

Tipps:

- Deklariere die Matrix in der richtigen Größe.
- Berechne Sigma.
- Berechne die Koordinaten der Mitte der Matrix.
- Berechne für jede Zelle der Matrix den Abstand r von Mitte mit dem Pythagoras. Setze den Wert der Zelle auf $e^{-\frac{r^2}{2\sigma^2}}$. (In Java e^x: Math.exp(x), $\sqrt{x}$: Math.sqrt(x), x^y: Math.pow(x,y))
- Berechne K, indem du alle Werte der Matrix addierst und den Kehrwert bildest.
- Multipliziere jede Zelle der Matrix mit K.



```
//
PImage neuesBild =
createImage(originalbild.width,originalbild.height,RGB);
```

```
//
double rot   = 0.0;
double gruen = 0.0;
double blau  = 0.0;
```

```
//
int xx = x - halb;
int yy = y - halb;
```

```
//
int laenge = filter.length;
int halb   = laenge / 2;
```

```
//
for (int i = 0; i < laenge; i++) {
```

```
//
for (int j = 0; j < laenge; j++) {
```

```
//
for (int x = halb; x < originalbild.width - halb; x++){
```

```
//
for (int y = halb; y < originalbild.height - halb; y++) {
```

```
//
rot   += filter[i][j] * red(pixelAlt[xx+i][yy+j]);
gruen += filter[i][j] * green(pixelAlt[xx+i][yy+j]);
blau  += filter[i][j] * blue(pixelAlt[xx+i][yy+j]);
```

```
//
if(rot < 0.0)      rot   = 0.0;
if(rot > 255.0)    rot   = 255.0;
if(gruen < 0.0)    gruen = 0.0;
if(gruen > 255.0)  gruen = 255.0;
if(blau < 0.0)     blau  = 0.0;
if(blau > 255.0)   blau  = 255.0;
```

```
//
pixelNeu[x][y] = color((int) rot, (int) gruen, (int) blau);
neuesBild.set(x,y,pixelNeu[x][y]);
```

```
return neuesBild;
```

```
public PImage faltung(PImage originalbild, double[][] filter) {
```

```
}
```

```
}
```



```
}
```

```

```

```
}
```

```

```

```
}
```

Arbeitsschritte der Programmerstellung:

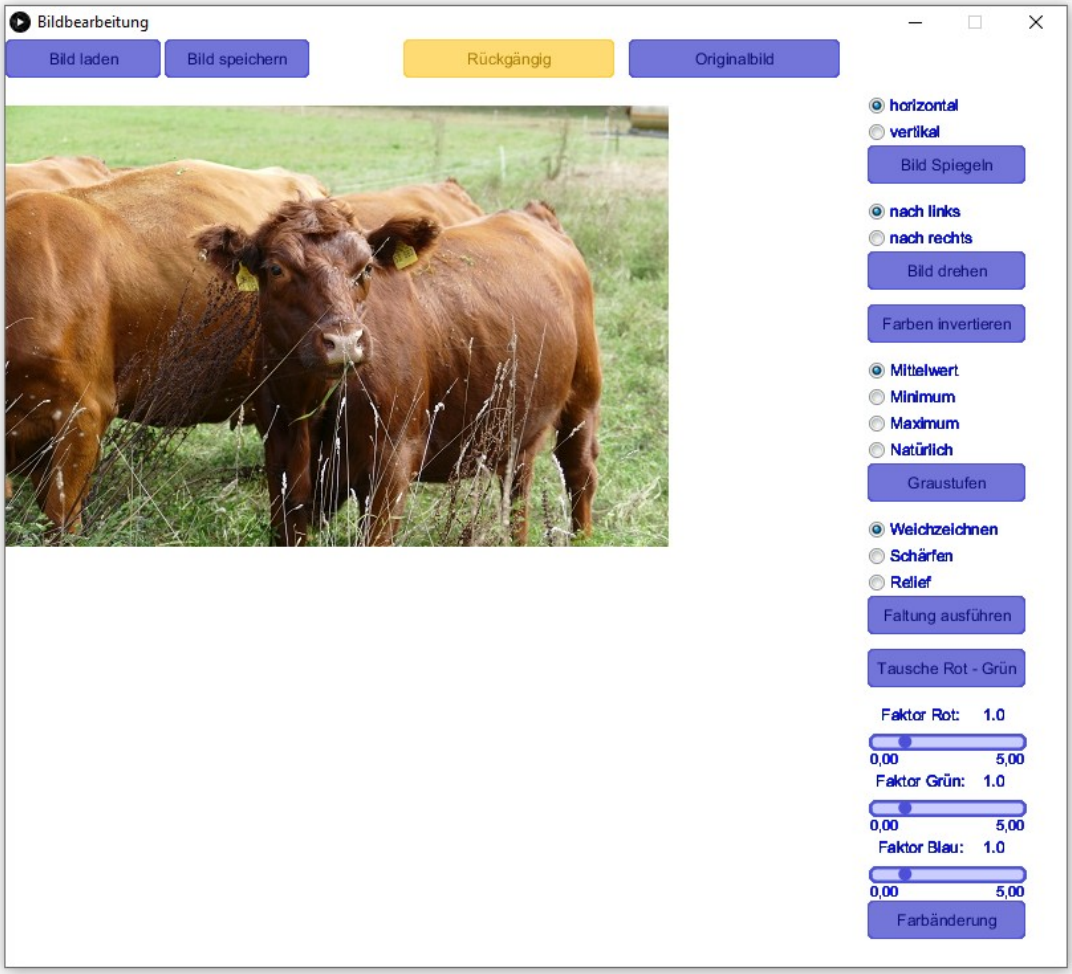
Nachdem die Algorithmen zur Bildbearbeitung implementiert und getestet wurden, kann nun die Programmoberfläche (Graphical User Interface) erstellt werden und mit den Algorithmen verknüpft werden. Daraus ergeben sich die drei Schritte:

1. Implementation und Testen der Algorithmen (**Model**)
2. Gestalten der Oberfläche mit einem GUI-Editor: hier G4P GUI Builder[1] (**View**)
3. Verknüpfen der GUI mit der Oberfläche: Implementation von Aktionen (**Controller**)

Die Trennung eines Programmierprojekts in diese drei Teile nennt man auch **Model-View-Controller (MVC)** Konzept und ist für größere Projekte Pflicht, da es die Übersicht erhöht. Die Interaktion zwischen den drei Teilen vereinfachen wir aber hier in der Schule gegenüber dem "echten" MVC-Konzept.

Gestaltung der Oberfläche

Am Ende könnte die Oberfläche in etwa so aussehen[2]:



1 In Processing: Hinzufügen über Tools – Tool hinzufügen: G4P GUI Builder auswählen und installieren

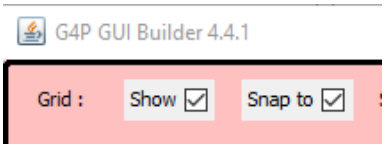
2 Alle Screenshots von eigenen Processing-Programmen, alle Bilder privat (Eisenmann)



Der G4P GUI Builder, der als Tool in Processing eingebunden werden kann, ist sehr einfach gehalten und ist deshalb auch einfach zu bedienen.

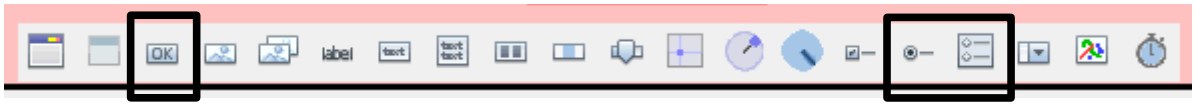
Erstellen der GUI

1. *Starte eine neue Processing-Datei und speichere sie unter „bildbearbeitung_v1“ ab.*
2. *Starte dann unter Tools den G4P GUI Builder.*
3. *In deinem Processing-Fenster wurden einige Zeilen Code automatisch eingefügt. Ändere in der setup()-Methode die Größe des Fensters auf 800 x 700 Pixel und lösche – sollte es in deiner Processing-Version vorhanden sein – den Import von „peasy“. Alles andere lässt du zunächst unverändert.*
4. *Stelle in G4P zunächst einmal das Gitter ein und lasse alles, was du einfügen wirst, am Gitter andocken.*



Für unsere Oberfläche brauchen wir Schaltflächen (Buttons) und die Möglichkeit, eine Auswahl zu treffen, um z.B. vor dem Klick auf eine Schaltfläche „Spiegeln“ auswählen zu können, ob horizontal oder vertikal gespiegelt werden soll. Für jede Auswahl nutzen wir eine Gruppe (ToggleGroup) mit mehreren Auswahlfeldern (Options). Die Gruppe regelt, dass immer nur eine Auswahl getroffen sein kann[3].

Eine Komponente wird einfach oben in der Leiste angeklickt und erscheint dann in der Mitte des Fensters. Von dort aus lässt sie sich mit der Maus an die gewünschte Stelle ziehen.



Ist eine Komponente ausgewählt, so kann man rechts unter „Properties“, wie der Name schon sagt, verschiedenen Eigenschaften der Komponente Werte zuweisen. Bei den Schaltflächen verändern wir den Namen der Komponente (Variable name), den Namen der Ereignismethode (Event method) und die Aufschrift (Text).

PROPERTIES	
GUI LOCK	☐
Variable Name	btSpiegeln
Event method	btSpiegelnClick
Text	Bild spiegeln

Zum Beispiel für die Schaltfläche zum Spiegeln des Bildes: „btSpiegeln“, „btSpiegelnClick“ und „Bild spiegeln“. Das „bt“ (oder auch „btn“) zu Beginn des Namens steht für die Art der Komponente, den Button. Das kennst du evtl. schon aus Klasse 8 vom MIT App Inventor. Das macht es beim Programmieren leichter zu erkennen, was sich hinter dem Namen versteckt.

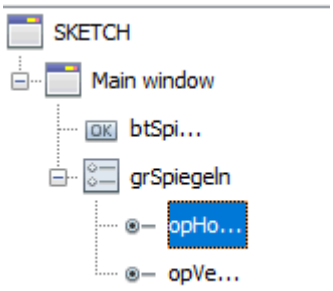
PROPERTIES	
GUI LOCK	☐
Variable Name	opHorizontal
Event method	
Selected?	☑
Display text	horizontal

Die Gruppe für die Auswahl nennen wir grSpiegeln („gr“ für group), die beiden zur Gruppe gehörenden Auswahl-Optionen nennen wir „opHorizontal“ und „opVertikal“. Da beim Klick auf eine Option, diese nur ausgewählt werden soll, aber ansonsten nichts passieren muss, leeren wir das „Event method“-Feld bei beiden und verändern nur den Text (Display text) zu „horizontal“ und „vertikal“.

3 Möchtest du noch Informationen über weitere Komponenten, kannst du auf der Seite des Programmieres Informationen und kleine Videosequenzen finden: *http://lagers.org.uk/g4ptool/index.html* (abgerufen am: 26.06.2020)

Die Komponenten sind auf dem Fenster sichtbar und werden gleichzeitig rechts hierarchisch aufgelistet:





5. *Gestalte deine Oberfläche mit Schaltflächen und Auswahlmöglichkeiten für alle Bildbearbeitungsarten, die du bisher schon programmiert hast. Achte dabei auf eine sinnvolle Namensgebung, wie oben beschrieben.*
6. *Starte dein Programm und schaue dir dein Werk an.*

Bevor wir uns jetzt um die Ereignismethoden kümmern können, brauchen wir die ganzen Bildbearbeitungsmethoden, die wir schon geschrieben haben.

7. *Kopiere aus deinen bisherigen Processing-Programmen alles Nötige für das Spiegeln des Bildes in das neue Programm mit GUI.*

Hier siehst du, wo dein Quelltext ergänzt werden muss:

bildbearbeitung_v1 | gui

```
1  // Need G4P library
2  import g4p_controls.*;
3
4  // ********* Deklarationen
5
6  public void setup(){
7    size(800, 700, JAVA2D);
8    createGUI();
9    customGUI();
10
11   // ********** setup()-Methode ergänzen, aber hier keine Bildbearbeitungs-
12   //            methode aufrufen!
13 }
14
15 public void draw(){
16   // background(230);  // wird hier auskommentiert, kommt in setup()
17 }
18
19 // ************* hier alle Bildbearbeitungsmethoden einfügen
20
21
22 // Use this method to add additional statements
23 // to customise the GUI controls
24 public void customGUI(){
25 }
```



Jetzt müssen wir noch die GUI mit den Aktionen verknüpfen. Indem wir beim Erstellen der GUI den Schaltflächen Ereignismethoden, wie z.B. btSpiegelnClick, zugewiesen haben, ist der erste Schritt schon erledigt: die Schaltfläche „btSpiegeln“ wartet ständig darauf, dass auf sie geklickt wird.

Wenn das der Fall ist, wird das ausgeführt, was in der Ereignisbehandlungsmethode „btSpiegelnClick“ geschrieben wurde. Um das müssen wir uns jetzt noch kümmern.

Den Rahmen dieser Methoden finden wir in dem automatisch mit der GUI erstellten zweiten Quelltext.

Achtung: Dort füllen wir nur genau diese Ereignismethoden. Alles andere schreiben wir weiterhin im Quelltext „bildbearbeitung_v1“.

Folgende Zeilen wurden automatisch eingefügt:

```
16
17 public void btSpiegelnClick(GButton source, GEvent event) { //_CODE_:btSpiegeln:781683:
18   println("btBildLaden - GButton >> GEvent." + event + " @ " + millis());
19 } //_CODE_:btSpiegeln:781683:
```

Beim Klick auf den Button wird bisher eine Information in der Konsolenzeile ausgegeben.

8. *Starte dein Programm, wie es bisher ist und klicke auf den Spiegeln-Button. Achte dabei auf die Ausgabe in der Konsole.*
9. *Kommentiere jetzt diese Zeile aus (oder lösche sie).*
10. *Überlege dir, was passieren soll, wenn auf den Button geklickt wird und formuliere es mit eigenen Worten.*

Falls „horizontal“ ausgewählt ist, dann soll das Bild horizontal gespiegelt werden, sonst vertikal.

Anschließend müssen wir das veränderte Bild wieder anzeigen, wie wir es bisher auch schon gemacht haben.

Die Methode könnte folgendermaßen aussehen:

```
public void btSpiegelnClick(GButton source, GEvent event) { //_CODE_:btSpiegeln:
  if (opHorizontal.isSelected()) {
    bildNeu = spiegleHorizontal(bildAlt);
  } else {
    bildNeu = spiegleVertikal(bildAlt);
  }
  background(255); // altes Bild wird übermalt
  image(bildNeu, 0, 0); // Bild wird an der Position (0|0) angezeigt
} //_CODE_:btSpiegeln:781683:
```

11. *Teste das Programm.*
12. *Ergänze eine Bildbearbeitungsart nach der anderen. Und mache erst weiter, wenn du getestet hast und mit dem Ergebnis zufrieden bist.*



Zusatzaufgaben:

1. *Informiere dich, wie ein sogenannter Slider funktioniert und baue für die drei Farben rot, grün und blau einen Slider ein, mit dem man zur Laufzeit deines Programmes die Farbanteile verändern kann. Nutze dann die Methode „farbaenderung()“.*
2. *Verändere dein Programm so, dass man ein neues Bild laden kann.*
3. *Verändere dein Programm so, dass man ein verändertes Bild speichern kann.*
4. *Schaffe eine Möglichkeit, eine Aktion rückgängig machen zu können.*
5. *Hole dir ein Arbeitsblatt zu einer weiteren Bildbearbeitungsmöglichkeit (Erweiterungen), die dir gefällt und bearbeite die Aufgaben.*

Pointillismus

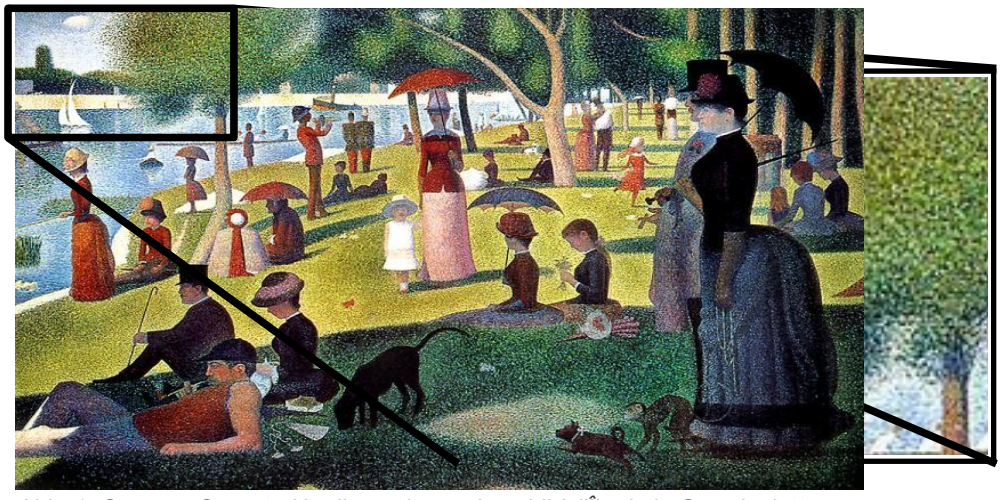
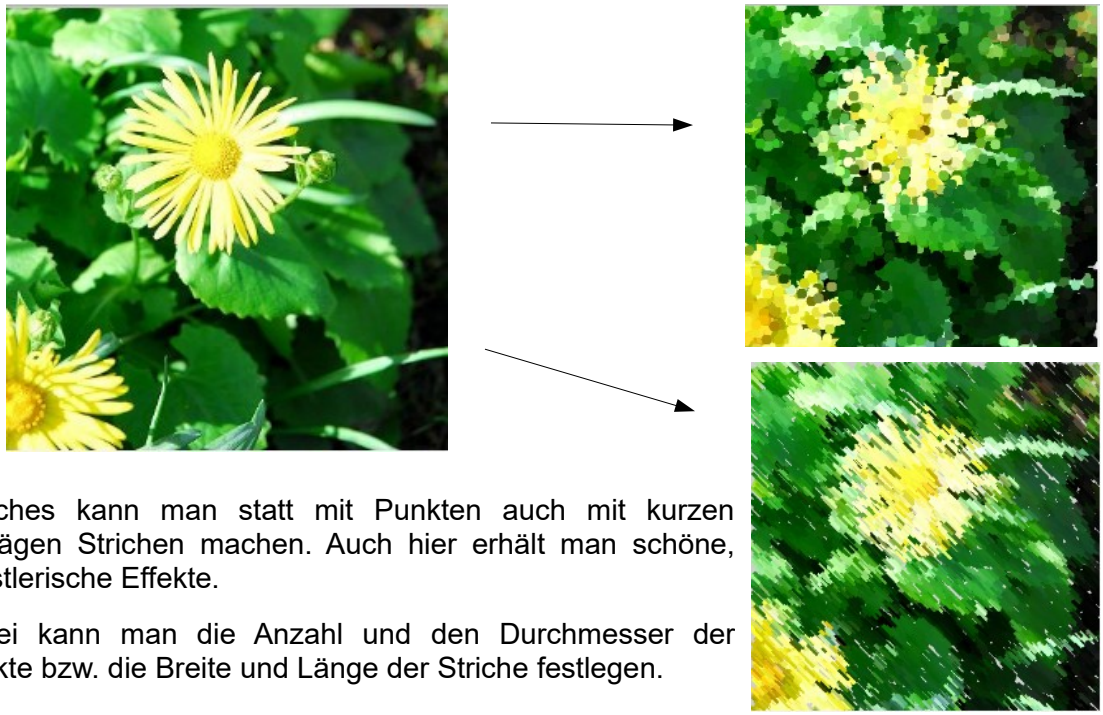


Abb. 1: Georges Seurat - Un dimanche après-midi à l'Île de la Grande Jatte Wikimedia Commons (Lizenz: Public Domain), URL: https://commons.wikimedia.org/wiki/File:Georges_Seurat_-_Un_dimanche_apr%C3%A8s-midi_%C3%A0_l%27%C3%8Ele_de_la_Grande_Jatte.jpg

Pointillismus bezeichnet eine Stilrichtung in der Malerei. Sie hatte in den Jahren zwischen *1889* und *1910* ihre Blütezeit. Der Pointillismus wird dem Post-Impressionismus zugeordnet.

Typisch für den Pointillismus ist der streng geometrisch durchkomponierte, oft ornamental wirkende Bildaufbau. Die Farben werden dabei in kleinen Punkten auf die Leinwand gesetzt. Dabei nutzten pointillistische Maler nur reine Farben. Farbmischungen entstehen erst auf dem Malgrund selbst.

Man kann einen ähnlichen Effekt erreichen, wenn man in einem Bild zufällige Pixel auswählt und dann ausgefüllte Kreise einer bestimmten Größe an diese Stelle zeichnet.



Gleiches kann man statt mit Punkten auch mit kurzen schrägen Strichen machen. Auch hier erhält man schöne, künstlerische Effekte.

Dabei kann man die Anzahl und den Durchmesser der Punkte bzw. die Breite und Länge der Striche festlegen.

Erweiterung: Stricheln / Tupfen



Zufallszahlen in Java

Um die Pixel zufällig auszuwählen, benötigt man Zufallszahlen. In Java gibt es eine Klasse Random, um Zufallszahlen zu erzeugen. Möchte man Sie benutzen, muss man sie importieren und dann ein Random-Objekt erzeugen. Bei diesem gibt es verschiedene Methoden, um reelle oder ganzzahlige Zufallszahlen zu erzeugen.

In Processing ist es etwas einfacher:

```
int x = int(random(originalbild.width));
```

erzeugt eine ganzzahlige Zufallszahl zwischen 0 und der Breite des Originalbildes.

Wiederholung: Zeichnen mit Processing

Die folgenden Zeichenmethoden kennst du schon aus Klasse 9:

```
stroke(250,0,0);      // Stiftfarbe rot
strokeWeight(3);      // Stiftbreite 3
line(10,10,30,10);    // Linie von (10,10) nach (30,10)
noStroke();           // kein Stift/Rand
fill(0,250,0);        // Füllfarbe grün
ellipse(20,20,10,10); // Kreisfläche an (20,20) mit d=10
```

Aufgaben

1. *Implementiere eine Methode*
   `PImage tupfen(PImage originalbild, int groesse, int anzahl)`
   Wähle eine zufällige x-Koordinate und eine zufällige y-Koordinate auf dem Bild aus. Setze die Füllfarbe auf die Farbe des Pixels im Originalbild und zeichne auf das Bild eine Kreisfläche (ohne Rand) an der ausgewählten Stelle mit dem Durchmesser `groesse`*. Wiederhole das in einer Schleife* `anzahl` *mal. Erzeuge danach ein neues Bild von der Größe des Originalbildes und übergebe jedem Pixel (wie gewohnt in einer verschachtelten Schleife) das Pixel der veränderten Oberfläche mit* `get(x,y)`*.*
   Teste deine Methode.
2. *Deklariere analog eine Methode* `stricheln` *mit den Parametern* `originalbild`*,* `dicke`*,* `laenge` *und* `anzahl`*.*
   Zeichne die gewünschte Anzahl von Strichen auf das neue Bild. Wähle auch hier die Positionen zufällig und benutze die Farbe des Originalbildes an dieser Stelle. Die Stiftbreite muss auf `dicke` *eingestellt werden.*
   Es sieht schöner aus, wenn die Striche schräg sind. Überlege dir, wie du den Endpunkt einer schrägen Strecke berechnen kannst, deren Länge über den Parameter verändert werden kann.

3. *Ergänze deine GUI so, dass man die nötigen Angaben für die beiden neuen Methoden über Textfelder eingeben und dann das geladene Bild tupfen oder stricheln lassen kann. (Info: Zahlen in Textfeldern gelten als String/Zeichenkette. Sie müssen umgewandelt werden. Z.B.* `int(tfStrichanzahl.getText())`*. Ist keine Zahl im Feld, erzeugt man einen Fehler.)*

Erweiterung: HSB-Farbmodell



HSB-Codierung

Das RGB-Modell ist für die Darstellung am Bildschirm gut geeignet, da ein Bildschirm aus winzigen roten, grünen und blauen Pünktchen besteht. Wir Menschen beschreiben Farben anders: Wir sagen z.B. helles / dunkles Rot, d.h. wir legen den Farbton (Hue) und die Helligkeit (Brightness) einer Farbe fest. Als dritte Komponente benötigen wir noch wie "rein" die Farbe ist - die Sättigung (Saturation). Dies führt zum HSB-Farbmodell (Hue-Saturation-Brightness).

Vom RGB-Modell kommt man zum HSB-Modell, wenn man den RGB-Würfel auf die Seite kippt, so dass Schwarz unten und Weiß oben ist. Die drei Primärfarben (R,G,B) und die drei Sekundärfarben (C,Y,M) befinden sich dann etwa auf halber Höhe. Man vereinfacht das zu einem Doppelkegel. Die Verbindungstrecke der beiden Spitzen sind die Grauabstufungen.

Die Helligkeit einer Farbe (Brightness) ist nun definiert durch die Höhe im Doppelkegel. Brightness 0 ist schwarz, 255 ist weiß. Der Farbton ergibt sich aus der Richtung, in der man sich von der Graustufenstrecke von Schwarz zu Weiß entfernt. Der Farbton wird (meistens) als Winkel ausgehend von der Rotrichtung angegeben (0° = Rot, 60° = Gelb usw). Die "Entfernung" von der Mittelachse ist dann die Sättigung einer Farbe. Sättigung 0 bedeutet grau, da man sich auf der Mittelachse befindet, Sättigung 255 bedeutet maximale Sättigung, d.h. man befindet sich auf der Mantelfläche des Doppelkegels.



Die Interpretation als Doppelkegel ist allerdings nur eine Möglichkeit, oft wird auch ein einfacher Kegel oder ein Zylinder verwendet. Die genauen Formeln für die Umrechnung[1] werden hier nicht benötigt, da Java Funktionen zur Umrechnung bereitstellt.

Aufgaben

1. *Gehe auf die Seite http://colorizer.org/ (Stand Jan. 2020). Experimentiere dort mit dem RGB- und HSB-Farbmodell:*
   - Welchen Wert haben Saturation und Brightness, wenn man eine Primär- (eine Farbkomponte 255, die anderen beiden 0) oder Sekundärfarbe (zwei Farbkomponenten 255, die dritte 0) im RGB-Farbmodell wählt?
   - Was passiert, wenn man auch die dritte Farbkomponente auf 255 erhöht?
   - Wie verändern sich die RGB-Werte, wenn man Sättigung (Saturation) und Helligkeit (Brightness) auf 100% einstellt und den Farbton (Hue) verändert? Wie verändern sie sich, wenn die Helligkeit nur auf 60% eingestellt ist?
   - Wähle eine beliebige RGB-Farbe. Stelle nun im HSB-Modell die Sättigung allmählich auf 0%. Wie verändert sich die Farbe? Wie verändern sich die RGB-Werte?
   - Wähle eine beliebige RGB-Farbe. Verändere nun die Helligkeit zwischen 0% und 100%. Wie verändern sich die RGB-Werte?

1 Formeln für die Umrechnung von RGB- in HSB-Farbraum findet man z.B. auf der Seite HSV-Farbraum (URL: https://de.wikipedia.org/wiki/HSV-Farbraum), Abrufdatum: 13.11.2019

Erweiterung: HSB-Farbmodell



HSB-Farbmodell in Processing

Processing arbeitet normalerweise mit dem RGB-Farbmodell. Will man mit dem HSB-Modell arbeiten, muss man den Farbmodus umstellen: `colorMode(HSB,360,100,100);`

Dabei wird jeweils der maximale Wert übergeben. Hier 360°, 100%, 100%.

Die einzelnen Werte erhält man über folgende Methoden:

```
color c = color(0, 100, 100);          // Erzeugt rote Farbe
float h = hue(c);                      // Liefert den Farbton von c
float s = saturation(c);               // Liefert die Sättigung von c
float b = brightness(c);               // Liefert Helligkeit von c
```

Aufgaben

2. *Implementiere die Methode* `PImage graustufenHSB(PImage originalbild)`



Verändere die Farbe über das HSB-Farbmodell jedes Pixels so, dass eine passende Graustufe gewählt wird. Hinweis: Dazu muss nur eine der Komponenten des HSB-Modells verändert werden.

3. *Füge in deiner GUI einen neue Auswahlmöglichkeit "Graustufen (HSB)" hinzu und ergänze den nötigen Code in der Button-Click-Methode.*



4. *Deklariere und implementiere eine Methode:*



```
PImage farbaenderungHSB(PImage originalbild,
                        float dHue, float fSaturation, float fBrightness)
```

Diese Methode soll die Farbe jedes Pixels so verändern, dass der Wert dHue zum aktuellen Hue-Wert addiert wird (d=Differenz). Die Werte fSaturation und fBrightness werden mit den bisherigen Werten multipliziert (f=Faktor). Beachte, dass der neue Hue-Wert über 360° hinausgehen darf, die Werte für Sättigung und Helligkeit 100(%) nicht überschreiten dürfen.

5. *Ergänze deine GUI so, dass man eine Farbanpassung vornehmen kann. Schaffe eine Möglichkeit, um die Differenz und die beiden Faktoren eingeben (Textfelder) oder auswählen (Schieberegler) zu können.*



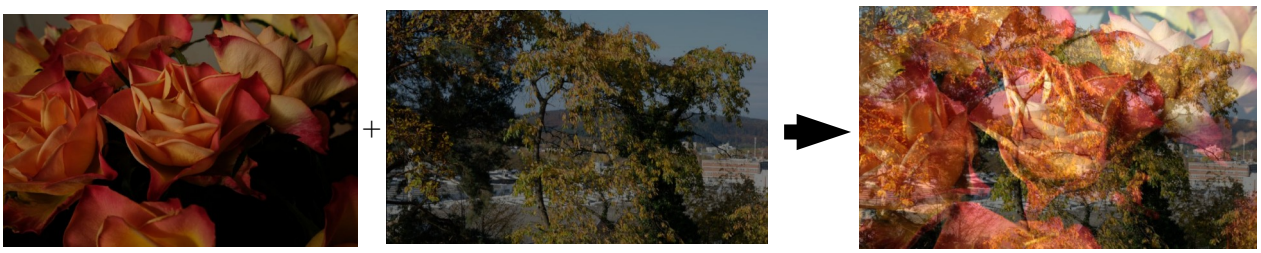
Bildkombinationen

In dieser Erweiterung geht es darum, zwei Bilder auf geeignete Weise zu kombinieren. Man kann die Bilder addieren, voneinander subtrahieren oder miteinander multiplizieren.

Addition

Bei der Addition zweier Bilder werden die Intensitäten der Pixel addiert. Dadurch entsteht eine Überlagerung zweier Bilder. Da jede Farbkomponente einen Wert zwischen 0 und 255 haben kann, kommen am Ende Werte zwischen 0 und 510 heraus. Alle Werte über 255 müssen auf 255 abgeschnitten werden.

Bei der Addition zweier normaler Bilder kommt aufgrund dieses Effekts ein Bild heraus, das insgesamt zu hell ist und sehr viele weiße Pixel hat. Denkbar wäre es, die Helligkeit der Bilder zunächst zu reduzieren.



Aufgaben

1. *Implementiere eine Methode* `PImage addition(PImage bild1, PImage bild2)`*, die für jedes Pixel des neuen Bildes die Intensitäten der jeweiligen Farbkomponente der beiden Bilder addiert. Gehe zunächst davon aus, dass die Bilder gleich groß sind. Beachte, dass nach der Addition der Intensitäten der Bereich 0-255 einzuhalten ist. Reduziere alle Werte größer 255 auf den Wert 255.*
2. *Falls die Bilder nicht gleich groß sind, kommt es zu Fehlern. Nimmt man als Bildgröße für das neue Bild die minimale Breite und die minimale Höhe, löst man dieses Problem. Passe deine Implementation entsprechend an.*
   Hinweis: Mit der Methode `min(zahl1, zahl2)` *kann man das Minimum zweier Zahlen bestimmen.*
3. *Ergänze deine GUI so, dass man Addition (später noch Subtraktion und Multiplikation) auswählen und dann zwei Bilder kombinieren kann. Das kombinierte Bild wieder gespeichert werden können.*
   Teste deine Methode mit geeigneten Bildern.
   Tipp: Vorher brauchst du die Möglichkeit, ein zweites Bild laden zu können. Arbeite mit Bild 2 und der Kombination unabhängig von deinen bisherigen Bildern bildAlt und bildNeu.

Subtraktion

Bei der Subtraktion von Bildern werden die Intensitäten voneinander subtrahiert. Durch die Differenzbildung der Intensitäten kann das Ergebnis negativ werden. Negative Werte werden zu Null, um im Definitionsbereich (0-255) zu bleiben.

Dabei ist es entscheidend, welches Bild von welchem abgezogen wird (siehe Abbildung: links unten = links oben - rechts oben; rechts unten = rechts oben - links oben).

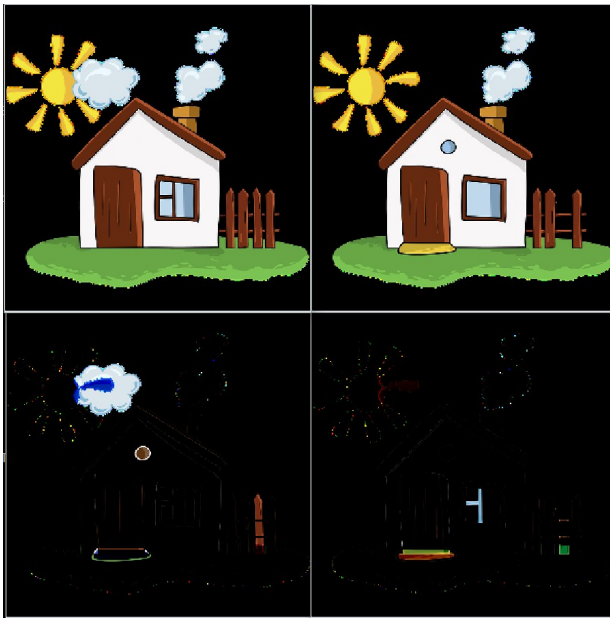


Abb. 1: Differenzbildung bei „Finde den Unterschied“, creozavr (Pixabay Lizenz) via Pixabay, URL: https://pixabay.com/de/illustrations/haus-häuser-finde-den-unterschied-3208132/ (17.11.19)

Aufgaben

4. *Implementiere eine Methode `PImage subtraktion(PImage bild1, PImage bild2)`, die für jeden Pixel des neuen Bildes die Intensitäten der jeweiligen Farbkomponente der beiden Bilder subtrahiert. Gehe zunächst davon aus, dass die Bilder gleich groß sind.*
   Beachte, dass nach der Subtraktion der Intensitäten der Bereich 0-255 einzuhalten ist. Ändere alle negativen Werte auf den Wert 0.
5. *Falls die Bilder wieder nicht gleich groß sind, kommt es zu Fehlern. Nimm wieder als Bildgröße für das neue Bild die minimale Breite und die minimale Höhe.*

6. *Ergänze die GUI um die weitere Auswahloption „subtrahieren“ (Ergänzung zu Aufgabe 3) und teste deine Methode zunächst mit den Bildern „unterschiedfinden1.png“ und „unterschiedfinden2.png“. Vertausche auch die Reihenfolge der Bilder.*
   Experimentiere dann: Lade die Bilder „baum1.jpg“ und „baum2.jpg“ als bild1 und bild 2. Subtrahiere zunächst bild2 von bild1 und speichere das Ergebnis als diff1. Subtrahiere dann bild2 von bild1 und speichere das Ergebnis als diff2. Addiere diff1 und diff2 und betrachte das Ergebnis.
   Beide Baum-Bilder wurden im Abstand von wenigen Millisekunden aufgenommen. Was kann man nun erkennen? Man kann den Effekt noch deutlicher machen, wenn man durch lineare Histogrammanpassung (Ergänzungsblatt 10) den Bereich auf 40 bis 100 einschränkt. Wie könnte man diese Operation für das autonome Fahren nutzen?
7. *Öffne die Datei "zonenplatte_cosinus.png". Wende mehrmals die Operation "Weichzeichnen" an, bis die dünnen Linien außen verschwinden. Weichzeichnen wird auch als "Low-Pass-Filter" bezeichnet, da schnelle Helligkeitsänderungen (dünne Linien) im Bild verschwinden und langsame Änderungen (dicke Linien) erhalten bleiben (passieren können). Speichere das Ergebnis als neue Datei.*
   Wende die Vorgehensweise aus Aufgabe 6 auf das Originalbild mit dem neuen Bild an. Beschreibe, was man nun sieht. Addiere zu diesem Bild das Originalbild. Beschreibe den Effekt, den alle diese Operationen zusammengenommen auf das Originalbild haben.



Multiplikation

Bei der Multiplikation von zwei Bildern werden die Intensitäten multipliziert. Durch die Multiplikation entstehen Werte zwischen 0 und 255^2 = 65025. Sinnvoller wäre hier eine Normierung der Intensitätswerte vor der Multiplikation auf einen Wertebereich von 0 bis 1. Dadurch bleibt der Wertbereich auch nach der Multiplikation erhalten. Danach wird der Bereich wieder auf 0-255 erweitert.

$$I_{neu} = \frac{I_{Bild\,1}}{255} \cdot \frac{I_{Bild\,2}}{255} \cdot 255 = I_{Bild\,1} \cdot I_{Bild\,2} \;/\; 255$$

Aufgaben

8. *Implementiere eine Methode* `PImage multiplikation(PImage bild1, PImage bild2)`*, die für jedes Pixel des neuen Bildes die Intensitäten der jeweiligen Farbkomponente der beiden Bilder multipliziert. Gehe zunächst davon aus, dass die Bilder gleich groß sind.*
   Beachte, dass nach der Multiplikation der Intensitäten der Bereich 0-255 einzuhalten ist. Wende daher die oben genannte Formel an.
   Teste deine Methode mit den Bildern "katze.jpg" und "maske1.jpg" bzw. "maske2.jpg".
   Erstelle mit einem Bildbearbeitungsprogramm eine eigene Maske.
   Erkläre, wie eine Maske funktioniert.
9. *Falls die Bilder wieder nicht gleich groß sind, kommt es zu Fehlern. Nimm auch hier wieder als Bildgröße für das neue Bild die minimale Breite und die minimale Höhe.*

10. *Ergänze in der GUI die Auswahlmöglichkeit „multiplizieren“ und teste deine Methode mit den Bildern „katze.jpg“ und „maske1.jpg“ bzw. „maske2.jpg“.*
    Erstelle mit einem Bildbearbeitungsprogramm eine eigenen Maske.
    Erkläre, wie eine Maske funktioniert.
11. *Wende eine Maske auf ein Bild an. Wende danach die Vorgehensweise aus Aufgabe 6 an, um Unterschiede zwischen dem Originalbild und dem veränderten Bild zu finden. Welchen Effekt hat das?*